Revision-Aware Security and Provenance Consistency Analysis of Public AI Models: A Preliminary Study

Ezekiel Ologunde

ABSTRACT

Public AI repositories combine documentation, loading configurations, code, and serialized weights. We implement a non-executing Python overlay that retains scanner reports and adds revision-bound, scoped consistency assessments. A seeded sample of 60 repositories from a download-ranked, 100-repository PEFT-tagged frame yields 59 pinned snapshots. Among 38 adapter configurations, 28 card/adapter declarations agree, two differ, six lack a card base, and two use unresolved local paths. The differing IDs admit compatible intermediate-base explanations rather than established false lineage. A separate developer-authored suite of 36 controlled cases contains eleven contradictions. The overlay detects eight with no false positives (precision 1.000, recall 0.727, F1 0.842), missing three paraphrases. Scope and identity checks eliminate designed false warnings. No supported explicit security claim is extracted from the real sample. The completed preliminary study establishes implementation conformance and identifies an observability bottleneck; it does not establish population contradiction prevalence, real-world accuracy, authenticated provenance, or improved malware detection.

KEYWORDS

model provenance, model cards, AI supply chain, security scanning, Hugging Face, consistency analysis

1 INTRODUCTION

AI model reuse introduces dependencies beyond source-code packages: upstream checkpoints, adapters, tokenizers, custom loader modules, training data declarations, and serialization formats. A repository may accurately describe an ancestral foundation model while an adapter configuration points to an intermediate loading dependency. It may offer Safetensors weights while retaining legacy pickle artifacts. These observations matter to reviewers, but neither observation alone establishes deception or malicious behavior.

Stalnaker et al. analyze 760,460 models and 175,000 datasets and motivate automatic inconsistency checking and validation [4]. Liang et al. examine 32,111 model cards and identify uneven documentation coverage [5]. These findings motivate connecting declarations to inspectable repository evidence. Security adds a distinct concern: Hugging Face documents code-execution risks in pickle deserialization [13], and PickleBall reports limitations in scanners and restrictive loading policies [12]. Its reported 44.9% pickle usage concerns its popular-model population; it is not an estimate for every Hub repository or for our sample.

We investigate a narrower question: can an analysis layer explain when a published claim conflicts with same-revision evidence, while abstaining when evidence or relationship semantics are insufficient? The implementation augments existing scanner reports rather than replacing a malware detector. Its units of analysis are explicit claims and the selected loading path. A filename is a screening signal, a byte-level format observation is stronger evidence, and a clean scanner result remains bounded by the scanner's coverage.

The completed experiment provides pinned evidence and acquisition receipts, an executable overlay, and controlled predictions with ablations and error analysis. The real cohort yields no initial security-grammar matches, and the cases share the rules' development process. The supported claim is implementation conformance; deployment effectiveness remains unvalidated.

2 RELATED WORK AND CANDIDATE GAP

DocML already connects model development and documentation through nudging and traceability [1]. DARA detects architecture-related naming and metadata inconsistencies [2]. These precedents rule out claiming the first model inconsistency detector. Our task instead concerns narrowly expressed security disclosures and loading dependencies in externally published repositories.

2.1 Documentation and provenance

Large-scale documentation analysis already measures missing fields, references, and licensing [4, 5]. PeaTMOSS links pretrained models with downstream software use and provides reusable mining infrastructure [6]. Unified model records address machine-readable genealogy and publication [7]. Weight-based heritage recovery, such as MoTHer, targets model ancestry using evidence beyond publisher metadata [8]. A configuration agreement cannot substitute for such lineage evidence.

Artifact-grounded documentation generation is also established. Alharbi et al. provide an artifact retrieval and enhanced-card pipeline [9]. We inspected the author repository and public preview; the publisher full text was unavailable in this review. MetaGAI evaluates multi-source model/data-card generation and triangulation [10]. Consequently, cross-source aggregation or a command-line interface alone is insufficient novelty.

The closest supply-chain work by Chen et al. examines transformations and dependency quality [11]; it overlaps a broad claim of checking declared bases. An existing public provenance scanner also implements provenance markers, hash checks, and temporal comparisons [15]. We inspected its source and assessment, but did not reproduce its performance. We do not claim the first provenance scanner or the first version-aware analysis tool.

2.2 Artifact security

Epyon combines pickle scanning with model-card compliance [3]. Its inspected Layer 15 checker evaluates headings, metadata and textual format recommendations. Combining documentation checks with artifact scanning is therefore established practice. We distinguish assessing whether a particular disclosure conflicts

with artifact evidence from checking documentation completeness. PickleBall generates and enforces library-specific loading policies [12]. PickleScan statically identifies suspicious globals in serialized files [14]. These systems address unsafe behavior or deserialization, whereas our target is agreement between a declaration and its scoped evidence. A correctly disclosed custom loader may introduce exposure without contradiction. Conversely, an unsupported lineage statement may be unverifiable without showing a loading threat. These targets require separate labels and denominators.

2.3 Candidate research gap

Within the targeted literature and implementation review, the candidate gap is an evaluated task for security-relevant, claim-level documentation/artifact consistency at an exact repository revision, with explicit abstention, loading-path scope, and incremental value over a fixed scanner workflow. This is a project-specific synthesis, not an exhaustive absence claim. The present experiment provides a small conformance benchmark and feasibility observations; a stronger novelty claim requires independent natural-case evaluation against the closest systems.

Our contribution is the operationalization and preliminary evaluation of this scoped task: a claim-centered overlay retaining scanner reports, an authored conformance suite, a frozen observability study, and a small differential probe. Revision pinning, hashing, traceability and uncertainty handling are reused safeguards, not individually novel mechanisms. The central follow-up question is what additional confirmed disclosure contradictions this overlay identifies beyond a fixed artifact-scanning and card-compliance workflow on independently annotated natural claims.

3 RESEARCH QUESTIONS AND HYPOTHESES

RQ1 (observability). In a defined repository sample, how often are declarations and compatible artifact evidence available, and how often do their observable identifiers agree?

RQ2 (security interpretation). Which assessments distinguish a documentation contradiction, a loading exposure, and insufficient evidence when claim scope and loading paths differ?

RQ3 (incremental detection). What additional contradiction assessments does the overlay produce beyond document-only checks, and how do scope and report identity affect errors in controlled cases?

H1. Artifact evidence increases contradiction detection relative to document-only analysis. The controlled suite tests the narrow engineering version. A deployment version would require independently annotated natural claims and a matched comparison.

H2. Loading-path scope reduces false exposure warnings relative to a blanket repository/filename rule while retaining recall. We report both false warnings and recall on the controlled suite.

H3 (exploratory, untested). Provenance contradictions are associated with independently established loading exposures. The present corpus lacks independent exposure labels, and the observed identifier disagreements do not establish provenance contradictions. Testing this hypothesis would be premature and potentially circular.

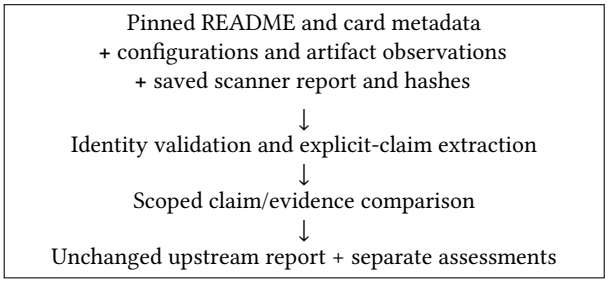


Figure 1: Non-executing overlay. Public metadata observations and accepted byte-bound scanner evidence retain different evidentiary strength.

4 EVIDENCE MODEL AND IMPLEMENTATION

4.1 Input and identity

The Python implementation accepts a local JSON snapshot with model ID, exact commit revision, original README, card metadata, configurations, inventory, and any verified artifact hashes/formats. A caller may specify an intended loader class and selected weight artifacts. This loading-path input is an explicit assumption, not an automatic proof of what every user will execute.

An optional scanner wrapper records scanner name/version, target revision, artifact SHA-256 values, validated format observations, and the original scanner result. The overlay accepts wrapper evidence only if the model and revision match, every supplied artifact hash matches the snapshot, and every format observation has a bound artifact hash. A missing report, missing hashes, stale revision, hash mismatch, or unbound format is recorded as a distinct binding status. The original input report is copied without altering its findings; accepted evidence is used only in the overlay namespace.

4.2 Checks and labels

The initial explicit-claim grammar recognizes serialization exclusivity, direct adapter loading dependencies, and assertions that custom Python code is unnecessary. It ignores quoted examples, fenced code as claim text, recommendations, and "not only" format statements. It is deliberately narrow and misses alternative phrasing.

For serialization, "Safetensors available" is different from "only Safetensors." A repository-level exclusivity claim conflicts with a confirmed pickle artifact in that scope. A loading-path claim instead considers only explicitly selected artifacts. Unknown bytes cause abstention; an extension alone is not confirmation. Source-code files are excluded from weight-format comparisons.

For adapter dependencies, a statement naming the required loading base is compared with `base_model_name_or_path`. Hub URLs are normalized; model-ID case is retained. Local paths remain unresolved. Generic card `base_model` metadata is compared separately: a differing ID is a review candidate because ancestry and immediate loading dependency may differ. A training-parent or merge-input statement is not converted into an adapter-load claim.

For custom code, a selected loader mapping, a documented `trust_remote_code=True` flag, and a present referenced local module support a conditional custom-code exposure assessment. Module presence or a trust flag by itself is insufficient. External module references and unavailable mappings can remain indeterminate. This rule is a limited static representation of a selected loader path, not a general Python call-graph analysis.

Consistency labels are *contradicted*, *consistent with observed evidence*, and *unverifiable*. A missing claim is counted separately from an unverifiable extracted claim. Exposure labels separately record observed loading exposure, no exposure observed within the inspected scope, or indeterminate. "Observed exposure" identifies a planned pickle/custom-code path; it is not an exploitation verdict and does not model every restrictive loader policy.

5 DATA AND EXPERIMENTAL METHOD

5.1 Author datasets and their roles

We downloaded and hashed the Stalnaker replication package, containing 760,460 normalized model records and 175,000 dataset records. The model records have identifiers, popularity, declared bases/datasets, architectures, and licenses, but no complete same-revision weight evidence or contradiction labels. They support historical profiling and sampling exploration rather than supervised contradiction training.

We also acquired the public PeaTMOSS SQLite sample (40 models, 225 Hugging Face commit records), PickleBall survey/import tables (1,703 traced-model rows and 2,312 model-information rows), a 50-model enhanced-card preview, and a 384-row supply-chain manual-check table. These resources informed the evidence schema and literature comparison. They were not pooled as interchangeable ground truth. PickleBall's large malicious/benign binary archives and the full PeaTMOSS repository corpus were not needed for this static preliminary experiment and were not downloaded. Licensing and redistribution conditions remain source-specific.

5.2 Fresh observational cohort

The recorded API query requests one page of 100 PEFT-tagged repositories, ranked by downloads. We sort that frozen frame by ID and sample 60 without replacement using seed 20261004. "PEFT-tagged" describes the retrieval frame; it does not guarantee that all selected repositories contain an adapter. The frame was captured on 4 October 2026 UTC. Selection is reproducible from the saved page, while a future live query may differ.

For each selected ID we use the listing's exact commit revision to request metadata and any available README, adapter configuration, model configuration, or merge recipe. We collect 59 repositories and record one unavailable target. We do not silently replace it. The available files are saved with retrieval URLs, timestamps, byte counts, and SHA-256 receipts. Weight bytes are not collected across this cohort, so pickle/Safetensors extension counts remain inventory signals.

The operational lineage comparison checks whether the adapter loading dependency appears among the card's declared base IDs. It has five observed outcomes: agreement, differing IDs, missing card base, unresolved local dependency, and no adapter configuration. We trace the two differing IDs to upstream public declarations for qualitative context; this is AI-assisted declaration inspection rather than independent human annotation or authenticated lineage recovery.

5.3 Controlled suite and baselines

The separate suite contains 36 unique developer-authored scenarios: 14 serialization, ten adapter-base, and twelve code/loading cases. Eleven contain a contradiction and eleven contain a defined selected-path exposure. These are synthetic documents and evidence snapshots; they are not mined natural contradictions. Labels follow the specified intervention and scope. The authorship of the rules and cases is not independent, there is no blinded test set, and no population inference is made from these counts.

The fixtures include a generated benign pickle, an inert pickle referencing `builtins.eval`, a minimal Safetensors artifact, opaque text with a `.bin` extension, and a custom-module text file. Static opcode inspection identifies pickle structure without calling `pickle.load`. PickleScan 1.0.5 scans the two pickle fixtures. The suspicious-global fixture is not described as a deployed malware campaign or a representative malicious-model corpus.

Five variants are compared on the same cases: (1) document-only analysis with configuration/artifact evidence removed; (2) the same checks with a naive pickle-extension heuristic and whole-repository exposure scope; (3) the full overlay; (4) the overlay without scope; and (5) the overlay without report identity enforcement. These are matched implementation baselines/ablations, not five independently engineered tools. In particular, the filename baseline is not PickleScan.

We measure case-level binary contradiction and exposure predictions, including TP, FP, FN, TN, precision, recall, and F1. No prediction on a positive case counts as a false negative. Extracted-claim abstentions and cases without an extracted claim are reported separately. Precision is undefined if there are no positive predictions. We use descriptive counts rather than statistical significance tests: the 36 scenarios were selected by design, not sampled independently from a deployment population.

5.4 Public artifact integration demonstration

A separate bounded demonstration downloads a 244,862-byte checkpoint from sshleifer/tiny-distilbert-base-cased-distilled-squad at commit `33a976c7`…. PickleScan reports one scanned file, zero issues, and no scan error. Static opcode inspection confirms consecutive pickle streams in its legacy Torch serialization. The wrapper binds the exact checkpoint hash and revision, and the overlay preserves the scanner result. This demonstrates report integration on public model bytes; one clean scan cannot establish malware-detection accuracy or universal safety.

5.5 Constructed comparator probe

After the initial study, we pinned the HF provenance scanner at commit `861d2f5e`… and Epyon at `c72bf76f`…. Two constructed cards share the same 35-byte benign pickle artifact and heading. One states "Weights are only Safetensors."; the other explicitly discloses pickle storage. We ran the HF scanner local CLI (version 0.2.0) on the two input files per case, with zero skipped files. We

Table 1: Observed card/adapter comparison outcomes.

Outcome	Repositories
No adapter configuration	21
Declarations agree	28
Card base missing with adapter present	6
Unresolved local adapter dependency	2
Differing declared IDs, requiring review	2
Total collected	59

also ran Epyon's original inline Python card checker with its shell variables replaced by literal paths; its shell wrapper and pickle layer were not executed. The overlay receives the fixture's known format and selected loading path. Tool input/output contracts differ, so we do not assign a shared accuracy score. These post-review probes are not held out.

6 RESULTS

6.1 Comparator probe: disclosure versus artifact risk

The HF local scanner emits no findings for either benign-artifact case. Epyon's card component emits the same eight documentation check names for both. The overlay reports one contradiction for false Safetensors exclusivity and zero for the accurate pickle disclosure. This illustrates an additional disclosure finding even when artifact analysis finds no suspicious behavior. A scanner need not flag a benign pickle to satisfy its own purpose. Two authored examples establish neither competitor incapability across modes nor deployment superiority.

6.2 RQ1: Availability and declaration agreement

All 59 collected repositories have a README; 53 declare a card base, 38 have an adapter configuration, and eleven have a model configuration. Table 1 partitions all 59 snapshots. Among the 30 cases with comparable Hub IDs and declarations, 28 agree and two differ (2/30, 6.7%). This denominator excludes missing and unresolved cases and does not estimate false-lineage prevalence.

Inventory screening finds 20 repositories with pickle-like filenames, 41 with Safetensors-like filenames, and 16 with both. Two model configurations contain `auto_map`. These are presence signals, not confirmed formats, selected loading exposures, or maliciousness labels.

The explicit security-claim grammar extracts zero supported claims from the 59 cards. Therefore, zero automated contradictions in this cohort is a coverage result, not evidence that all cards are consistent. Structured base metadata remains observable, but the initial prose grammar cannot support a natural-case security-contradiction accuracy estimate.

6.3 Tracing the differing IDs

The nomic-ai/colnomic-embed-multimodal-3b card declares Qwen/Qwen2.5-VL-3B-Instruct, while its adapter configuration names vidore/colqwen2.5-base. The tsystems/colqwen2.5-3b-multilingual-v1.0 card declares the same Qwen model, while its adapter names tsystems/colqwen2.5-3b-base. Both upstream loading dependencies themselves declare that Qwen base in separately pinned public metadata.

These observations are compatible with intermediate loading checkpoints and ancestral card declarations. We retain the discrepancies and trace receipts without establishing deception or maliciousness. Relationship semantics constrain string comparisons; a compatible declared chain still does not authenticate historical training.

6.4 RQ2 and RQ3: Controlled detection and ablations

Table 2 reports contradiction results. The full overlay detects eight of eleven positives and produces no false positives. The document-only variant makes no positive predictions because these constructed positives depend on artifact/configuration evidence. This is a designed task separation, not a comparison against a strong general-language documentation model. The naive filename variant detects the same eight positives but introduces four false positives.

Removing scope creates one additional contradiction warning and five false selected-path exposure warnings. Removing identity enforcement creates two additional contradiction warnings and two false exposure warnings by accepting stale or wrong-hash reports. The full overlay preserves all 36 upstream inputs (including absent reports), with three non-null controlled report wrappers preserved. Seven focused regression tests pass for immutability, scope, local paths, identity, quotes, and ancestry semantics.

The full overlay records eight unverifiable extracted claims and nine cases without an extracted claim. Its three missed contradictions are distinct paraphrases: "safe tensor format" without the implemented exclusivity phrasing, attachment to a base "at inference time," and an assertion that only the standard-library implementation executes. We retain these misses rather than expanding the grammar to erase the benchmark errors. Since cases are developer-authored, this is not a holdout guarantee; it is an explicit account of current coverage.

The exposure rule detects all eleven constructed exposures without false positives. This perfect score is limited to the simplified known paths and fixtures in the suite. It does not validate general loader behavior, external custom code, scanner resilience, or malware identification. PickleScan's original benign/suspicious results remain unchanged; no malware-performance improvement is claimed.

6.5 Hypothesis status

The narrow controlled form of H1 is supported: artifact evidence enables eight additional correct contradiction detections over the implemented document-only ablation. The broader real-repository form is unresolved because no natural supported security claim is evaluated. H2 is supported within the designed cases: false exposure warnings fall from nine under the filename/whole-repository baseline to zero, while recall remains 11/11. H3 is untested. No

Table 2: Contradiction detection on 36 developer-authored cases (11 positive, 25 negative). P: precision; R: recall. "–" denotes undefined precision. These are conformance results, not real-world accuracy estimates.

Variant	TP	FP	FN	P	R	F1
Documentation only	0	0	11	–	0.000	0.000
Filename heuristic	8	4	3	0.667	0.727	0.696
Full overlay	8	0	3	1.000	0.727	0.842
Without scope	8	1	3	0.889	0.727	0.800
Without binding	8	2	3	0.800	0.727	0.762

Table 3: Selected-path exposure results on the same controlled cases (11 positive, 25 negative).

Variant	TP	FP	FN	Recall
Documentation only	0	0	11	0.000
Filename heuristic	11	9	0	1.000
Full overlay	11	0	0	1.000
Without scope	11	5	0	1.000
Without binding	11	2	0	1.000

causal association between provenance discrepancies and security exposure is established.

7 DISCUSSION AND THREATS TO VALIDITY

7.1 What the experiment changes

The observability bottleneck constrains the task more strongly than controlled precision: this popular PEFT-tagged sample yields no matching security phrases. Future sampling should measure natural loading assurances and variants. An assurance-rich stratum can support error analysis; a probability-sampled stratum is needed for prevalence estimates.

Equating every card base with an immediate adapter dependency would turn the two observed ambiguities into unjustified warnings. Authenticating provenance requires stronger evidence, such as training records, attestations, or independent weight analysis.

7.2 Construct and internal validity

The contradiction target is observable disagreement for the same stated relationship and scope, not maliciousness or actual training origin. The code rule represents a conditional selected loader path and does not execute or fully resolve Python control flow. Format observations in fixtures are known structural checks; filenames in the observational cohort remain unverified. Unknown cases do not imply safe cases.

The controlled cases and rules share a developer process. The suite emphasizes intended invariants and may favor the implementation. Three difficult paraphrases expose a real extraction limit, but do not remove authorship bias. The protocol is saved for reproducibility and is not a preregistration. A local-path defect found during development was corrected and covered by a regression test before the reported run. Runtime, if reproduced, is a single in-memory run rather than a stable performance benchmark.

7.3 External and statistical validity

The cohort is a random subset of one popularity-ranked, PEFT-tagged API page. It excludes less popular repositories, inaccessible artifacts, and other loading ecosystems; tagging may include converted or adapter-free repositories. It is not a representative sample of all public AI models. The one unavailable target is retained as missing. Historical replication records and fresh snapshots have different collection times and are not treated as longitudinal evidence.

No independent annotators establish real-case contradiction or exposure labels, no inter-rater agreement is reported, and no real precision/recall is estimated. Counts in the authored suite have no random-sample confidence interpretation. A larger final study should freeze an annotation guide, independently annotate flagged and unflagged natural cases, adjudicate disagreement, and separate publishers/families and copied cards across development and evaluation. Metrics should include extraction, assessability, abstention, and security relevance, not just F1 on detected claims.

8 REPRODUCIBILITY AND DATA HANDLING

The full local artifact records three pinned comparator source inventories with 180 verified file hashes, original outputs for the two disclosure probes, the novelty audit and prediction-blinded review materials. Independent natural-case labels remain pending. A public controlled-only supplement provides the original CLI/core, case builder, seven unit tests and constructed reference outcomes at https://github.com/ezekielologunde/ai-security-artifact-audits/tree/main/reports/P02. It excludes natural model cards, metadata snapshots, downloaded weights, comparator source and reviewer material; it does not independently reproduce the natural cohort. The full offline archive remains private. The local artifact provides acquisition scripts, the frozen API frame and selected IDs, revision-pinned evidence, receipts, the CLI, fixture bytes, controlled inputs, scanner wrappers, predictions, summaries, and error analysis. Running scripts/run_experiment.py uses saved local data; it does not refresh the frame. The acquisition script caches existing evidence. Refreshing collection intentionally changes the study snapshot and must be treated as a new run.

A retrospective release check on 20 October 2026 reexecuted the full offline experiment in an isolated copy: observational counts, all controlled predictions and error cases matched; only elapsed runtime was excluded from summary comparison. A separate controlled-only wrapper reproduced its reference outcomes. The claim-level review supplement requires reviewer-built assertion

inventories, but no independent annotations are complete. These checks establish reproducibility rather than natural-case accuracy.

The completed experiment uses Python 3.12, PickleScan 1.0.5, and pinned dependencies. SHA-256 verification covers 179 downloaded experiment files, totaling 1,797,888 bytes, with no recorded mismatch. These receipts establish local byte integrity against the saved manifest; they do not authenticate publishers. The separate public-artifact demonstration preserves the exact checkpoint and scanner report for reproduction.

No model code is imported, no downloaded weights are deserialized, and no large model is trained. Human-subject data are not collected. Public repositories are not equated with universally open-source licensing. Author datasets and cards retain their original terms; a shareable code/results archive references source evidence without assigning a blanket license to third-party material. The paper is an author draft in ACM two-column layout, with no claim of ACM acceptance or publication.

AI assistance. Generative AI assisted literature discovery, code development, manuscript drafting and qualitative declaration tracing. No AI-generated assessment is presented as an independent human label. The named author is responsible for checking the sources, artifacts and conclusions before public submission.

9 CONCLUSION

The implemented overlay connects explicit claims with scoped, revision-bound evidence and retains upstream scanner results. The completed preliminary study documents 59 public repository snapshots, two identifier discrepancies compatible with intermediate dependencies, and 36 controlled cases. It demonstrates useful scope/identity invariants and exposes three grammar misses. It also finds no supported explicit security claims in the sampled real cards, which prevents a real-world detection-effectiveness claim. The next scientific requirement is independent natural-case evaluation and broader claim extraction; the present artifacts make that requirement concrete and reproducible.

REFERENCES

[1] Avinash Bhat, Austin Coursey, Grace Hu, Sixian Li, Nadia Nahar, Shurui Zhou, Christian Kästner, and Jin L.C. Guo. 2023. Aspirations and Practice of ML Model Documentation: Moving the Needle with Nudging and Traceability. In *CHI 2023*. https://doi.org/10.1145/3544548.3581518

[2] Wenxin Jiang, Mingyu Kim, Chingwo Cheung, Heesoo Kim, George K. Thiruvathukal, and James C. Davis. 2025. "I see models being a whole other thing": an empirical study of pre-trained model naming conventions and a tool for enhancing naming consistency. *Empirical Software Engineering* 30, 155. https://doi.org/10.1007/s10664-025-10711-4

[3] MetroStar. 2026. Epyon: Model Card Compliance Checker. Inspected and probed commit `c72bf76f`…. https://github.com/MetroStar/epyon/tree/c72bf76f556fc5b253580c81b4535db20914c1c6

[4] Trevor Stalnaker, Nathan Wintersgill, Oscar Chaparro, Laura A. Heymann, Massimiliano Di Penta, Daniel M. German, and Denys Poshyvanyk. 2025. An Empirical Analysis of Machine Learning Model and Dataset Documentation, Supply Chain, and Licensing Challenges on Hugging Face. *arXiv:2502.04484v2*. https://arxiv.org/abs/2502.04484

[5] Weixin Liang, Nazneen Rajani, Xinyu Yang, Ezinwanne Ozoani, Eric Wu, Yiqun Chen, Daniel Scott Smith, and James Zou. 2024. What's documented in AI? Systematic Analysis of 32K AI Model Cards. *arXiv:2402.05160*. https://arxiv.org/abs/2402.05160

[6] Wenxin Jiang, Jason Jones, Jerin Yasmin, Nicholas Synovic, Rajeev Sashti, Sophie Chen, George K. Thiruvathukal, Yuan Tian, and James C. Davis. 2024. PeaTMOSS: Mining Pre-Trained Models in Open-Source Software. *MSR 2024; arXiv:2310.03620*. https://arxiv.org/abs/2310.03620

[7] Keyu Wang, Abdullah Norozi Iranzad, Scott Schaffter, Meg Risdal, Doina Precup, and Jonathan Lebensold. 2024. Mitigating Downstream Model Risks via Model Provenance. *arXiv:2410.02230v2*. https://arxiv.org/abs/2410.02230

[8] Eliahu Horwitz, Asaf Shul, and Yedid Hoshen. 2025. Unsupervised Model Tree Heritage Recovery. *ICLR 2025; arXiv:2405.18432*. https://arxiv.org/abs/2405.18432

[9] Nuha Alharbi, Emad Albassam, and Samar Alkhuraiji. 2026. Enhancing AI Transparency on Hugging Face: An Automated Pipeline for Model Card Generation and Reproducibility Scoring From Repository Artifacts. *IEEE Access*. https://doi.org/10.1109/ACCESS.2026.3723027

[10] Haoxuan Zhang, Ruochi Li, Yang Zhang, Zhenni Liang, Junhua Ding, Ting Xiao, and Haihua Chen. 2026. MetaGAI: A Large-Scale and High-Quality Benchmark for Generative AI Model and Data Card Generation. *arXiv:2604.23539*. https://arxiv.org/abs/2604.23539

[11] Ziqian Chen, Zekai Chen, Susheng Wu, Bihuan Chen, Wenyan Song, Yiheng Huang, Zhuotong Zhou, Yiheng Cao, and Xin Peng. 2026. A First Look at Model Supply Chain: From the Risk Perspective. In *Proceedings of ICSE 2026*. ACM, 12 pages. https://doi.org/10.1145/3744916.3773171

[12] Andreas D. Kellas, Neophytos Christou, Wenxin Jiang, Penghui Li, Laurent Simon, Yaniv David, Vasileios P. Kemerlis, James C. Davis, and Junfeng Yang. 2025. PickleBall: Secure Deserialization of Pickle-based Machine Learning Models (Extended Report). *arXiv:2508.15987v2*. https://arxiv.org/abs/2508.15987

[13] Hugging Face. 2026. Pickle Scanning. Hub documentation (accessed October 2026). https://huggingface.co/docs/hub/security-pickle

[14] PickleScan contributors. 2026. PickleScan: Security scanner detecting Python Pickle files performing suspicious actions. Version 1.0.5 used in this study. https://github.com/mmaitre314/picklescan

[15] Provenance scanner contributors. 2026. HF Model Provenance Scanner. Local CLI probed at commit `861d2f5e`…; earlier review at `251a7b90`…. https://github.com/poojakira/hf-model-provenance-scanner